BME 133 Lab 0

Welcome! This is a Jupyter/IPython notebook, hence the file extension ".ipynb"

Our labs for this class will typically come in the form of a Jupyter notebook. There are two types of cells in a Jupyter Notebook, Markdown (for formatted text) and Code. This is a Markdown cell; double click on it to reveal the underlying Markdown. To view the formatted text again, click the checkmark in the upper right-hand corner of this cell.

In [ ]:

In [ ]:
```
# This is a Python code cell, notice it has a 'play' button on the left and
# Go ahead and press the play button, or press Shift+Enter on your keyboard,
print("Hello World")
```

Hopefully you are viewing this file on your computer using Visual Studio Code on an Anaconda kernel, but if you have trouble with installation, today you can use Jupyter's browser-based platform. In the long-run, it's better to use VSCode because it has more resources than Jupyter alone, and running code locally on your own computer is more secure in terms of data and privacy.

To begin, create a copy of this file using File -> Save as... and name the new file "Lab_0_LastName_FirstName.ipynb" using your last and first name.

This Lab will cover Variables, basic Data Types, Operators, and Commenting

Variables

Variables in Python can be used to assign values to characters. Assign the variable name to the left side, and put the value on the right side. Use an equal sign to perform the assignment.

- Note: Variable names are case sensitive.

In [5]:
```
x = 1  # assign the integer 1 to the variable x
y = '2'  # assign the string '2' to the variable y
z = "3"  # you can also use double quotes for strings

print(x)
print(y)
print(z)
```

```
1
2
3
```

Variable assignments are saved in memory between cells in the notebook (as long as the kernel has not been restarted).

In [6]:
```
print(y)
```

```
2
```

If you re-use the same variable name later, the initial assignment will be replaced.

In [7]:
```
y = '4'  # replace the previous assignment of '2' with '4'
print(y)
```

```
4
```

Variable names should be descriptive and make sense in the context of your pipeline.

In [8]:
```
apple_color = 'green' # Assign the variable apple_color the value 'green'
print(apple_color)
```

```
green
```

Executing this next cell won't work. Numbers can't be at the start of the variable. Notice what happens when you try to execute this.

In [9]:
```
1st_variable = '43110'
print(1st_variable)
```

```
  Cell In[9], line 1
    1st_variable = '43110'
    ^
SyntaxError: invalid decimal literal
```

You can also print strings while including your variable within the string. Here are three possible ways.

In [10]:
```
# Old Way, somewhat confusing
print('The value of z is %s' % z)

# New Way #1
name = 'Bob'
print('Hello, {}'.format(name))

# New Way #2
a = 5
b = 10
f'Five plus ten is {a + b} and not {2 * (a + b)}.' # This method is called a
```

```
The value of z is 3
Hello, Bob
```

Out[10]:
```
'Five plus ten is 15 and not 30.'
```

Data Types

Variables can store different types of data. Some of the most common types are:

- Text is called a string or: `str`
- Integers (whole numbers): `int`
- Floating point numbers (has a decimal point): `float`
- Boolean (True/False): `bool`

You can determine what data type a variable contains using the `type()` function.

In [11]:
```
x = 3
print(type(x))
```

```
<class 'int'>
```

In [12]:
```
y = 3.14
print(type(y))
```

```
<class 'float'>
```

In [13]:
```
z = True
print(type(z))
```

```
<class 'bool'>
```

You can change the data type using casting (or type casting) functions such as `int()`, `str()`, or `float()`

In [14]:
```
w = float(x)
print(w)
print(type(w))

v = int(y)

print(v)
print(type(v))
```

```
3.0
<class 'float'>
3
<class 'int'>
```

Watch out for floating point errors which occur due to the use of binary in hardware. Some decimals cannot be expressed exactly as binary fractions

In [15]:
```
0.1 + 0.2
```

Out[15]:
```
0.30000000000000004
```

Operators

Operators in Python perform operations on variable and values. They come in many varieties including: Arithmetic, Assignment, Comparison, and Logical (we'll address

logical operators and other types later)

See the full list of operators at W3Schools.

Arithmetic Operators

In [16]:
```
x = 2

# Examples
print(x + 5) # This operator is for addition
print(x - 3) # This operator is for subtraction
print(x * 4) # This operator is for multiplication
print(x / 4) # This operator is for division
print(x ** 2) # This operator is for exponentiation
print(x // 2) # This operator is for floor division
```

```
7
-1
8
0.5
4
1
```

The Modulo (also called modulus) operator `%` returns the remainder of a given equation.

In [17]:
```
print(5 % 2)
```

```
1
```

Comparison Operators

- Note the kind of output comparison operators produce.

In [18]:
```
# Examples
print(x == 2) # This operator means 'is exactly equal to'
print(x != 2.1) # This operator means 'is not equal to'
print(x < 3) # This operator evaluates whether the magnitude of x is less th
print(x >= 3) # This operator evaluates whether the magnitude of x is greate
```

```
True
True
True
False
```

Assignment Operators

In [19]:
```
# Examples
y = 5 # This operator assigns the variable name y to the value 5
y += 1 # This operator increases the value of y by the specified amount
y *= 2 # This operator multiplies the value of y by the specified amount
print(y)
```

```
12
```

Comments

In [ ]:
```
# This is comment, comments do not affect the code, but are important for pr

"""
This is a mutiline comment.
Multiline comments are especially useful in explaining how functions work,
but that's a story for another time.
"""
```

In [ ]:
```
# In VSCode, you can quickly comment out multiple lines of code by highlight
# You can uncomment code highlighting it and pressing Ctrl+K+U (Mac: Commanc
# Try running this cell, then uncomment the code and run the cell again.

# a = 10
# b = 5
# eqn = (a * b)
# result = (eqn > 25)
# print(f'The statement that a*b is > 25 is: {result}')
```

Exercises

Excercise 0

Before running the next code cell, guess what the output will be.

In [21]:
```
x = 1
y = '2'
z = "3"

# What do you think output will be?
print(y + z)
```

```
23
```

Was the output what you expected? Why do you think it gave the result that it did?

In [ ]:
```
# Answer here: 23. Yes. Because your concatenating two strings
```

Excercise 1

What do you think will happen?

In [3]:
```
print(x + int(y))
```

```
3
```

What do you think the function `int()` did to the variable `y` ?

In [ ]:
```
# Answer here: 3. Converts the y string to an integer. so 1 + 2 = 3.
```

Excercise 2

Use a casting function to convert the data type of variable `w` into a floating point number.

Check your result using the `type()` function.

In [4]:
```
# Answer below:
w = float(24)
print(w) #print the value of w
print(type(w)) #Use the type() function to print the type of w
```

```
24.0
<class 'float'>
```

Excercise 3

What is the difference between the operators `*` and `*=` ?

In [23]:
```
# Answer here: "*" multiples two operands together. Multiples itself by the
```

Excercise 4

Write an expression that prints True if the integer is even, and False if the interger is Odd. Hint: use the modulo operator `%` to return the remainder.

`6 / 2` returns `3` ; division
`6 % 2` returns `0` ; meaning that the remainder is 0

In [ ]:
```
#Answer here: print(6 % 2 == 0) returns true
# print(5 % 2 == 0) returns false
```

In [35]:
```
print(6 % 2 == 0)
print(5 % 2 == 0)
```

```
True
False
```

Getting to know you 🤩

Leave a comment here stating whether you have any coding experience (if so, what language(s)?), and what you hope to gain from this class.

In [ ]:
```
#Answer here: Rohit - Yes. Two years as a professional software developer ir
# Neha - C++ and basic python. Intermediate for c++.
# Dania - MatLab - intermediate
```

```
# We all want strengthen our python skills to help and gain practical experi
```

In [ ]:

In [ ]: